

Guión de presentación

TP3 — Perceptrones

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Persona	Bloque	Slides	Tiempo
P1	Intro + Ej0 (resultados de validación)	1–6	5:30
P2	Ej1 — Distillation: MSE, ROC, threshold sweep	7–12	5:30
P3	Ej2 — Sweeps, learning curves, final_eval test	13–19	5:30
P4	Ej3 — Pack C: tabla, comparison, análisis del techo	20–26	5:30

Total: 22 minutos (más Q&A).

Foco: resultados, plots y gráficos y por qué nos dio cada cosa. No hablamos de arquitectura del código, ni de tests, ni de implementación interna; eso queda para el informe técnico si preguntan.

Convenciones:

- **0:30** marca dónde debería estar el orador a los 30 segundos del bloque.
- **[Slide N]** indica cuándo cambiar de diapositiva.
- **/** pausa breve (respirar). **//** pausa más larga (cambio de tema).
- *texto cursivo rojo* = referencia explícita al enunciado o apunte.
- **texto en azul** = identificador del repo cuando aparece (mínimo).

Índice

1. Persona 1 — Introducción + Ej0 (validación)	1
2. Persona 2 — Ej1 — Knowledge Distillation	3
3. Persona 3 — Ej2 — Clasificación de dígitos	5
4. Persona 4 — Ej3 + Conclusiones	6
Apéndice — preguntas anticipadas	8

1. Persona 1 — Introducción + Ej0 (validación)

Foco del bloque: mostrar qué pide el TP, los resultados de la validación con AND y XOR (los dos plots de convergencia y la métrica final), y dejar la transición lista para el ejercicio 1.
Volumen: ~ 800 palabras a 145 wpm.

Apertura — 0:00 – 0:25

[Slide 1, portada]

Buenas. / Trabajo Práctico tres, Perceptrones. / Somos cuatro y nos vamos a turnar en orden de los ejercicios. Yo arranco con la introducción y los resultados del ejercicio cero, que es la validación. Después siguen los otros tres, uno por ejercicio. Al final dejamos espacio para preguntas.

Qué pide el TP — 0:25 – 1:10

[Slide 2, qué pide el TP]

El enunciado tiene tres bloques de problemas y una validación previa. // La validación es el ejercicio cero: probar que el perceptrón escalón resuelve el AND, y que un MLP resuelve el XOR. / El ejercicio uno es detección de fraude por knowledge distillation. / El ejercicio dos es clasificación de dígitos manuscritos con un MLP, y el ejercicio tres es lo mismo que el dos pero apuntando a un *accuracy mayor o igual a noventa y ocho por ciento sobre el test set*.

Y la regla del juego más importante para el resto de la presentación: *el set de prueba, digits_test.csv, se evalúa una sola vez al final*. No se toca durante búsqueda de hiperparámetros. Eso va a ser clave cuando lleguemos a los resultados del ejercicio dos.

Validación AND con escalón — 1:10 – 2:10

[Slide 3, AND con step perceptron]

Vamos a la validación. // El primer caso es el AND lógico con un perceptrón de función escalón. Lo hacemos en *representación bipolar como pide el apunte*: las entradas son menos uno o más uno, no cero o uno.

// Resultado: el perceptrón clasifica las cuatro muestras del AND con error cero en pocas epochs. / Esa es la prueba mínima: con dos clases linealmente separables y un solo umbral, la regla delta clásica converge.

// Esto importa porque establece la línea base. / La función escalón es la primera versión del perceptrón, la que vio Rosenblatt en los años cincuenta. Y ya en este caso uno ve algo interesante: si rompemos el AND, no hay forma de que el resto del TP funcione. Por eso lo dejamos como test *antes* de los modelos grandes.

XOR con MLP: el resultado clásico — 2:10 – 3:50

[Slide 4, XOR con MLP — convergencia]

Y acá viene el resultado más famoso del campo, el que motivó toda la teoría del aprendizaje profundo a partir del paper de Minsky y Papert.

// El XOR no es separable linealmente. Un perceptrón simple, una sola capa, no puede resolverlo.
/ Pero con una capa oculta no lineal, sí. / En el plot ven dos curvas, las dos arquitecturas que probamos, en escala logarítmica de loss. // La azul es la mínima: **dos-dos-uno**, dos entradas, dos neuronas ocultas, una salida. / La verde es **dos-tres-dos-uno**, con una capa oculta extra de tres neuronas.

Ambas convergen. / La de dos-dos-uno tarda más en bajar el loss — noten la pendiente al principio — pero termina en el mismo orden de magnitud que la otra. / Eso es lo que demuestra el resultado clásico: dos neuronas ocultas alcanzan, una capa más solo acelera la convergencia.

// Detalle: ambas redes usan tanh como activación de las hiddens, MSE como loss, y SGD con learning rate cero coma uno. Esos son los hiperparámetros del config que usamos.

Lo que la validación nos dice del engine — 3:50 – 4:50

[Slide 5, qué validamos]

¿Qué nos dice esto sobre el engine que vamos a usar para los ejercicios siguientes? // Tres cosas concretas.

// Una. / La pipeline de *forward pass* y backpropagation está bien implementada, porque el XOR converge a loss casi cero. Si hubiera un bug en backprop, eso no pasaría.

Dos. / Las activaciones tanh y sus derivadas están bien definidas. Probamos con tanh porque *el apunte la usa como activación canónica para problemas con salida en menos uno y más uno*.

Tres. / El MSE como función de loss y su gradiente también están bien.

// En total son tres componentes que vamos a reusar tal cual en los ejercicios dos y tres. Cambia la activación final a softmax, el loss pasa a cross-entropy, pero la mecánica de backpropagation es la misma. La validación con XOR nos da confianza para escalar a problemas grandes sin gastar tiempo debuggeando.

Cierre y transición — 4:50 – 5:30

[Slide 6, transición]

Para cerrar. // El AND y el XOR son los dos casos de validación que pide el enunciado. / El AND lo resolvió un perceptrón simple con error cero. El XOR lo resolvieron dos arquitecturas distintas de MLP, con resultados consistentes.

// Esa validación nos da una pipeline confiable para los ejercicios siguientes. / El primero de esos es detección de fraude, donde no usamos un MLP sino un perceptrón simple porque la salida del modelo objetivo es continua y unidimensional. / Le paso a quien lo presenta.

Wordcount estimado P1: ≈ 815 palabras. A 145 wpm $\rightarrow 5 : 37$. Margen para pausas. **Si llegás corto:** agregar un comentario sobre el rol histórico del XOR en la teoría del perceptrón. **Si llegás largo:** simplificar la sección "qué validamos.^a un solo párrafo.

2. Persona 2 — Ej1 — Knowledge Distillation

Foco del bloque: mostrar la comparación lineal vs no lineal por MSE, la curva ROC, la PR curve, el threshold sweep, y la recomendación final basada en el costo asimétrico.

Volumen: ~ 800 palabras.

El problema en una frase — 0:00 – 0:40

[Slide 7, problema]

Yo me ocupo del ejercicio uno. // El problema en una frase: tenemos un BigModel pre-entrenado que clasifica fraude, y nuestra tarea es entrenar un perceptrón simple que replique su salida. Eso se llama *knowledge distillation*.

// La salida del BigModel es continua, una probabilidad de fraude entre cero y uno. / Los inputs son nueve features de la transacción: monto, duración de sesión, edad de la cuenta, y otros parecidos. / Y hay una columna adicional, `flagged_fraud`, con la etiqueta binaria de fraude real, que *el enunciado prohíbe usar para entrenar*: la usamos solo para evaluar después con métricas operativas.

Lineal vs no lineal: el plot del MSE — 0:40 – 1:50

[Slide 8, MSE comparison]

Probamos las dos versiones que pide el enunciado. // El perceptrón lineal — que es Adaline — y el perceptrón no lineal con activación sigmoide. / Mismo dataset, mismo K-fold cinco estratificado, mismas features.

// Resultado: / MSE del lineal, **cero coma cero dos seis seis**. / MSE del no lineal, **cero coma cero uno uno cero**. / **Reducción del cincuenta y nueve por ciento**.

¿Por qué gana la sigmoide tan claro? // El target del distillation está acotado entre cero y uno: es una probabilidad. / La sigmoide *naturalmente* mapea su output a ese rango. / El lineal puede producir valores negativos o mayores a uno, y el MSE penaliza esa salida fuera del dominio. Si el modelo predice uno coma dos cuando el target es cero coma nueve, paga un error grande aunque ordene bien la predicción.

// Eso es el primer take-away del ejercicio: **cuando el target tiene un dominio acotado, la activación final tiene que respetarlo**.

El threshold default es engañoso — 1:50 – 2:50

[Slide 9, threshold default]

Ahora viene la parte interesante. // Si tomamos el modelo no lineal y aplicamos el threshold default — cero coma cinco — las métricas operativas son malas. / Precision, treinta y tres por ciento. Recall, cien por ciento. / Es decir, clasifica casi todo como fraude.

// Eso a primera vista suena a modelo terrible. Pero el modelo no es el problema. *El threshold no está calibrado al modelo*.

¿Por qué? / Porque las salidas sigmoide se concentran en la parte alta. / La mediana de los scores de las muestras de fraude está en cero coma noventa y nueve. / Y los no-fraudes no se van a cero coma cero cinco como uno esperaría: están en cero coma cuatro o cero coma seis. / El modelo aprendió un buen *ranking*, pero las probabilidades absolutas no están calibradas. Y con threshold bajo, eso se traduce en sobre-detección.

Threshold sweep: ROC, PR y la elección — 2:50 – 4:30

[Slide 10, threshold sweep]

La solución es barrer el threshold. // En este plot del sweep ven precision en azul, recall en verde, F1 en negro, todo en función del threshold de cero a uno.

// A la izquierda, threshold cero: precision igual a la prevalencia de fraude, recall uno. Clasifica todo como fraude. / A la derecha, threshold uno: precision uno, recall casi cero. No clasifica casi nada. / El óptimo por F1 está **en threshold cero coma ochenta y nueve**: precision ochenta y nueve, recall ochenta y seis, F1 ochenta y siete.

[Slide 11, ROC y PR]

Acá las dos curvas que faltaban. // A la izquierda, la curva ROC: tasa de verdaderos positivos contra falsos positivos. El área bajo la curva, AUC-ROC, es **cero coma noventa y nueve dos uno** para el modelo no lineal, y cero coma noventa y siete dos para el lineal. / Ese AUC tan alto indica que la *capacidad de discriminación global* del modelo es excelente. El problema solo es dónde cortar.

// A la derecha, la curva precision-recall. Esta es más informativa que la ROC para un problema con clases desbalanceadas como fraude. / Veán cómo precision se mantiene alta hasta recall ochenta por ciento, y después cae rápido. Eso quiere decir que hay un punto de operación natural ahí.

Recomendación de umbral — 4:30 – 5:30

[Slide 12, recomendación]

Para cerrar el ejercicio uno: // ¿qué umbral recomendamos al cliente? / Depende del costo asimétrico entre falso positivo y falso negativo.

// **Opción A:** threshold cero coma ochenta y nueve. Maximiza F1. Detecta el ochenta y seis por ciento de los fraudes reales con once por ciento de falsos positivos. Esta es la opción si la auditoría manual es barata.

// **Opción B:** threshold cero coma noventa y cinco. Precision pasa de noventa y cinco por ciento, pero recall baja a setenta y cinco. Esta es la opción si los falsos positivos son caros y el cliente prefiere no alertar de más, aunque pierda algunos fraudes reales.

// El modelo entrega scores. La política de threshold la define el dueño del producto. / Le paso al siguiente para el ejercicio dos.

Wordcount estimado P2: ≈ 800 palabras. A 145 wpm $\rightarrow 5 : 30$. **Si llegás corto:** agregar un comentario sobre la prevalencia de fraude en el dataset (clase positiva $\approx 12\%$). **Si llegás largo:** no detallar la opción B, solo mencionar que "para clientes con auditoría cara, threshold $\geq 0,95$ ".

3. Persona 3 — Ej2 — Clasificación de dígitos

Foco del bloque: contar el problema, mostrar los plots de los sweeps de Fase 1 y 2, las learning curves del base, la matriz de confusión, y el cachetazo del final_eval con su explicación.
Volumen: ~ 800 palabras.

El problema y los datos — 0:00 – 0:40

[Slide 13, problema]

Yo cuento el ejercicio dos: clasificación de dígitos manuscritos del cero al nueve. // Cada muestra es una imagen de veintiocho por veintiocho pixeles, en escala de grises, aplanada en un *vector de setecientos ochenta y cuatro floats entre cero y uno*. / El train tiene doce mil cuatrocientos cincuenta muestras. El test, dos mil quinientos. / Y como dijo el primero, **el test set se evalúa una sola vez, al final**.

// La arquitectura final, después de todos los sweeps que voy a mostrar, es: setecientos ochenta y cuatro entradas, cien neuronas en la primera capa oculta con ReLU, cincuenta en la segunda con ReLU, y diez salidas con softmax. Loss cross-entropy.

Workflow Fase 1 + Fase 2 — 0:40 – 1:30

[Slide 14, workflow]

Para no probar hiperparámetros al azar, dividimos la búsqueda en dos fases.

// **Fase uno, exploratoria.** K-folds igual a uno: un solo train-val split ochenta-veinte. / Sweeps secuenciales. Primero arquitectura, después optimizador, después learning rate, después batch size. / Cada sub-fase usa el ganador del anterior. Eso reduce el espacio de búsqueda enormemente.

// **Fase dos, validación.** K-fold igual a cinco. / Tomamos el config ganador de Fase uno y variamos un solo hiperparámetro a la vez para ver si las elecciones se sostienen con K-fold completo.

Sweeps de Fase 2: los plots — 1:30 – 3:00

[Slide 15, sweep LR]

Estos son los cuatro plots de los sweeps de Fase dos. // El primero es learning rate. / Vean los extremos: con learning rate igual a diez, la red queda en accuracy random, doce por ciento. Con cero coma uno, también diverge. / El óptimo está en cero coma cero cero uno. Confirma la elección de Fase uno.

[Slide 16, sweep arquitectura]

Sweep de arquitectura. / Probamos cuatro versiones alrededor del ganador. / Una shallow, con solo treinta neuronas en una capa: claramente subajustada. / La base de cien-cincuenta. / Una wider, doscientas-cien. / Y una deeper, cien-cincuenta-veinticinco-diez.

// Resultado interesante: / wider gana en val por cero coma diecisiete puntos sobre la base, pero tarda *tres veces y media* más en entrenar. / Empate técnico, mantenemos la base.

[Slide 17, sweeps opt e init]

Sweep de optimizador. / SGD, Momentum, Adam. / Adam y Momentum empatan dentro del desvío. SGD claramente atrás. / Y sweep de inicializador: He, Xavier y uniform. Empate total. La inicialización no es un cuello de botella.

// El take-away de Fase dos es interesante: **en general, los sweeps validan la elección de Fase uno más que descubrir un ganador nuevo.** La metodología coarse-to-fine ya hizo el grueso del trabajo.

Curvas y matriz de confusión del base — 3:00 – 4:00

[Slide 18, curvas y matriz]

Estos son los dos plots del modelo base con K-fold cinco. // A la izquierda, las curvas de aprendizaje. / Ven cómo train loss baja rápido y val loss se estabiliza alrededor de la época cinco. Convergencia limpia, sin overfitting visible.

// A la derecha, la matriz de confusión. / Diagonal limpia para nueve de las diez clases. La excepción — y esto es importante — **es la clase ocho**: precision cero, recall cero. / La red nunca predice ocho. Ya nos dio una pista de que algo raro pasaba con la distribución del dataset. Quédense con esto.

El cachetazo del final_eval — 4:00 – 5:30

[Slide 19, final_eval]

Y entonces hicimos el final eval. / Entrenamos con todo `digits.csv` y evaluamos una sola vez contra `digits_test.csv`. // Resultado.

// Val K-fold cinco: **noventa y seis coma veintidós por ciento.** / Test sobre `digits_test`: **ochenta y seis coma treinta por ciento.** // Drop de diez puntos porcentuales.

Eso fue una sorpresa fuerte. / La matriz de confusión sobre el test set, que ven a la derecha, confirma que las confusiones se reparten por todo el grid: no es una clase específica que falla. // El patrón es **distribution shift**. / El modelo está bien. *Los datos no se parecen.*

// La interpretación es que `digits.csv` y `digits_test.csv` no son muestras de la misma distribución. Quizás distintos escritores, quizás distintos estilos. Hay un agujero de cobertura del train que el test expone. / Si eso es cierto, lo que necesitamos no es un modelo más grande. *Necesitamos más datos que cubran la distribución del test.* / Y *eso es lo que el ejercicio tres nos provee*. Le paso al siguiente.

Wordcount estimado P3: ≈ 805 palabras. A 145 wpm $\rightarrow 5 : 33$. **Si llegás corto:** expandir el comentario sobre la clase ocho colapsada con la hipótesis de sub-representación. **Si llegás largo:** no detallar los cuatro sweeps de Fase 2 uno por uno: agruparlos en "todos confirman la elección excepto LR".

4. Persona 4 — Ej3 + Conclusiones

Foco del bloque: mostrar el +10pp del primer movimiento, la tabla de Pack C, el comparison plot Ej2 vs Ej3, el análisis honesto de qué movió la aguja y qué no, las hipótesis del techo, y las conclusiones del TP.

Volumen: ~ 800 palabras.

Hipótesis y plan — 0:00 – 0:50

[Slide 20, plan]

Yo cierro con el ejercicio tres y las conclusiones del TP. // La hipótesis que dejó la persona anterior es que el problema es distribution shift, no falta de capacidad. / Si eso es cierto, lo primero que hay que probar es *más datos*, no un modelo más grande.

Y eso es justamente lo que el enunciado del ejercicio tres provee: un dataset adicional, `more_digits.csv`, con quince mil setecientas muestras. / El plan fue secuencial. / **Primero**, probar con más datos sin tocar el modelo. / **Segundo**, si no alcanza, activar Pack C: la regularización del enunciado, que son L2, dropout, augmentation gaussiana y lr scheduling. / **Tercero**, si nada llega, escalar capacidad.

El primer dominó — 0:50 – 1:50

[Slide 21, el primer dominó]

El primer movimiento fue agregar `more_digits.csv`. / **Cero cambios al modelo**, solo concatenamos los dos CSVs. // Y el resultado fue impresionante.

// Val K-fold cinco subió de noventa y seis coma veintidós a noventa y siete coma cero seis. / Pero lo importante está en el test: **de ochenta y seis coma treinta a noventa y seis coma treinta y seis**. // **Diez puntos porcentuales en test sin tocar el modelo.**

Eso confirmó la hipótesis. / Y además, miren esto: macro F1 saltó de cero coma ochenta y cinco a cero coma noventa y cinco. / ¿Se acuerdan de la clase ocho que estaba colapsada en el ejercicio dos? / Ahora se predice correctamente. Más datos cubrieron el agujero.

La tabla de Pack C — 1:50 – 3:00

[Slide 22, tabla Pack C]

Pero faltaba uno coma sesenta y cuatro puntos para llegar al noventa y ocho. Así que arrancamos con Pack C.

// Esta tabla muestra todas las configs que probamos, ordenadas. / Cada fila es una técnica o combinación, cada columna es val K-fold cinco y test. / Vamos por la columna de test.

L2 con weight decay uno por diez a la menos cuatro: noventa y seis coma cincuenta y seis. Cero coma veinte sobre el baseline. / **Dropout solo, probabilidad cero coma dos:** noventa y seis coma veintiocho. *Peor* que el baseline. / **L2 más dropout combinados:** noventa y seis coma cero cuatro. Aún peor. // La fila resaltada en verde, **L2 más augmentation gaussiana con sigma cero coma cero cinco: noventa y seis coma ochenta y ocho.** / Ese es el ganador.

// Los demás abajo: l2 con augmentation y sigma más alto, baja un poco. / Las tres regularizaciones combinadas, peor que solo L2 con aug. / Y wider más L2 más aug, peor todavía.

Comparación visual Ej2 vs Ej3 — 3:00 – 3:40

[Slide 23, comparación visual]

Este es el plot de comparación entre el ejercicio dos y el ejercicio tres ganador. // Las cinco métricas de test, lado a lado. / La línea roja punteada es el target del noventa y ocho por ciento.

// El salto principal está en accuracy, F1 y recall. / Pasamos por arriba del noventa y seis en todo, pero queda casi un punto y medio para el target.

Qué movió la aguja, qué no — 3:40 – 4:30

[Slide 24, qué ayudó qué no]

Acá la parte más importante para defender los resultados. El análisis honesto.

// **Lo que ayudó.** / Más datos: diez puntos. El movimiento dominante por lejos. / L2: cero coma veinte puntos. / Augmentation gaussiana: cero coma treinta y dos puntos *adicionales* sobre L2. Hay sinergia entre L2 y aug, pero augmentation sola no alcanza.

// **Lo que no ayudó.** / Dropout. Y este caso es interesante: dropout gana en val, pierde en test. / Eso quiere decir que reduce varianza específica al fold, pero no al shift entre train y test. / La arquitectura wider, con doscientas neuronas: mejor en val, peor en test. Más capacidad memoriza el train. Eso descarta que el problema sea capacidad. / Y combinar todo a la vez: L2 más dropout más augmentation, peor que L2 más aug solo. La regularización compite consigo misma cuando se exagera.

Por qué no llegamos al 98 — 4:30 – 5:00

[Slide 25, hipótesis del techo]

Y la pregunta incómoda. // ¿Por qué no llegamos al noventa y ocho? / Mejor: noventa y seis coma ochenta y ocho. Faltan uno coma doce puntos. Tres hipótesis ordenadas por probabilidad.

// **Una.** Augmentation insuficiente. / El gaussian noise es *isotrópico*: ruido independiente por píxel. Pero el shift entre train y test parece ser de *estilo de escritura*: rotación pequeña, traslación, grosor de trazo. / Eso pide augmentation por transformaciones *geométricas*, no por ruido. No la implementamos: gaussian noise era la primera aproximación del enunciado.

// **Dos.** Sub-representación en el val interno del final eval, que usa val random del diez por ciento para early stopping. / Si los digits difíciles del test no están en ese diez por ciento, la red para antes de tiempo. Posible mejora: stratified val.

// **Tres.** Capacidad. Improbable, porque wider sobreajustó.

Conclusiones del TP — 5:00 – 5:30

[Slide 26, conclusiones]

Tres ideas para llevarse de todo el TP. // **Una.** Una buena pipeline gana más que tunear hiperparámetros: el más diez puntos del ejercicio tres vino de *datos*, no de modelo. / **Dos.** Los empates técnicos son mucho más comunes que los wins claros. Hay que elegir por parsimonia, no por cero coma cero cinco puntos. / **Tres.** Mejor en val no es mejor en test cuando hay shift. Dropout fue el ejemplo de manual.

// Cumplimos todo lo del enunciado excepto el target del noventa y ocho. Está documentado honestamente con hipótesis verificables. / Gracias. / Preguntas.

Wordcount estimado P4: ≈ 810 palabras. A 145 wpm $\rightarrow 5 : 35$. **Si llegás corto:** agregar un comentario sobre los costos de computar (Fase 2 con K-fold tardó horas). **Si llegás largo:** cortar la hipótesis tres sobre capacidad, ya está implícita en "wider sobreajustó".

Apéndice — preguntas anticipadas

¿Por qué no usaron sklearn o PyTorch?

El enunciado lo prohíbe explícitamente. La consigna pide perceptrones desde cero con NumPy.

¿Por qué dos fases en el ejercicio dos?

Para no caer en sweep aleatorio. Fase uno con K-folds igual a uno explora rápido y reduce el espacio. Fase dos con K igual a cinco valida con K-fold completo. Sin Fase dos, una elección que ganó por suerte en un solo fold parece definitiva.

¿Por qué no llegan al noventa y ocho? (versión hostil)

Identificamos que el techo viene de la calidad del augmentation, no de la capacidad del modelo. Probamos las cuatro técnicas Pack C que sugiere el enunciado y siete combinaciones distintas. Lo que faltaba implementar es augmentation por transformaciones geométricas: rotación pequeña, traslación. Es un follow-up explícito.

¿Por qué eligieron threshold cero coma ochenta y nueve en el ejercicio uno? (P2)

Maximiza F1 en el sweep. Si el cliente prefiere precisión sobre recall, threshold mayor a cero coma noventa y cinco. Es una decisión de negocio sobre el costo asimétrico falso positivo y falso negativo.

¿Cuántas muestras tiene el dataset de fraude? ¿Cuál es la prevalencia? (P2)

Aproximadamente nueve mil muestras totales con cinco-fold, prevalencia de la clase positiva alrededor del doce por ciento. Por eso la curva PR es más informativa que la ROC para este problema.

¿Qué arquitectura final usaron en el ejercicio tres? (P4)

La misma que el ejercicio dos: setecientos ochenta y cuatro, cien, cincuenta, diez. ReLU en las hidden, softmax en la salida. La regularización ganadora fue L2 uno por diez a la menos cuatro y augmentation gaussiana sigma cero coma cero cinco.

¿Por qué dropout no funcionó en el ejercicio tres? (P4)

Dropout reduce varianza dentro de la distribución del train. Acá el problema no es varianza dentro del train: es shift entre train y test. Dropout puede ayudar en val K-fold, donde las clases son las mismas que en train, pero no transfiere a un test que viene de una distribución parcialmente distinta.